ppmi-dbs-pain — code walkthrough for human readers

Niels Pacheco-Barrios

2026-05-19

Purpose of this document

This walkthrough explains what every script in `ppmi-dbs-pain/` does, why it exists, and the key lines you should understand before modifying it. The aim is comprehension, not exhaustive line-by-line transcription — where logic is repetitive, we summarize and point you to canonical examples.

Pair this with:

- `callgraph_overview.mmd` (Mermaid) — high-level flow.
- `outputs/figures/Figure_callgraph.png` — full callgraph (295 nodes).
- `outputs/figures/Figure_causal_DAG.png` — causal assumptions DAG.
- `docs/dashboard.html` — interactive results.

A printable PDF is produced by `make docs-pdf` (pandoc).

Repo orientation

```
ppmi-dbs-pain/
  README.md                  ← start here
  Makefile                   ← entry-point (make all)
  Dockerfile + .devcontainer ← one-click reviewer reproduction
  config.example.yml         ← copy to config.yml, fill in paths
  R/
     helpers/
        pain_helpers.R           ← canonical utilities
        make_synthetic_cohort.R  ← seeded fake-data generator
     build_*.R                    ← main analysis scripts
     25_genetics_arm_pain.R       ← exploratory genetic interactions
     26[abc]_pain_motor_*.R       ← pain-motor coupling
  sprints/sprint01-09.R       ← 9 post-hoc robustness scripts
  scripts/
     _build_methods_figure.py
     _build_docx_v9_*.py
     build_callgraph.py
     build_sankey.py
     build_dashboard.py
  notebooks/                  ← 36 Jupyter notebooks (original pipeline)
  tests/testthat/             ← unit tests
  data-synth/                 ← synthetic PPMI fixture
  ppmiTTE/                    ← extracted R package
  docs/                       ← Quarto book + this walkthrough
```

Helpers — `R/helpers/pain_helpers.R`

The single most important file. Anything that touches data goes through this module.

What it does

- Loads PPMI data (real or synthetic) via `config.yml`-driven paths.
- Defines colour palettes (Okabe-Ito + paper-original Tol Bright).
- Constructs anchor variants (DBS surgery date, cohort-median, patient first visit, symmetric midpoint).
- Provides save helpers that write PNG, PDF, and TIFF in one call.

Key sections

Lines	Section	Why it matters
8–11	`library()` block	All shared dependencies live here.
13–17	`here::i_am()` + `config.yml`	Replaces the old hardcoded `PROJECT_ROOT` path; reproducibility-critical.
25–35	`OKABE_ITO` palette	Default for all sprint figures; never use raw hex elsewhere.
47–80	`load_full_ppmi_rel_patient_anchors()`	Builds the analytic long frame with each patient's first visit as the anchor for Never-DBS, first DBS date for DBS arm.
81–101	`load_full_ppmi_rel()`	Same as above but with a *fixed* cohort-median Never-DBS anchor (legacy comparator).
270–290	`compute_symmetric_midpoint_anchors()`	**Critical.** Casts POSIXct to Date *before* arithmetic to avoid the "seconds-not-days" bug that previously collapsed midpoints to first_visit.
305–325	`rebind_time_cols()`	Rebuilds `time_days` / `time_months` / `months` / `time_bin` from a fresh anchor table.
326–350	`save_fig_pub()`	Writes PNG + PDF (and optional TIFF) in one call.
351–370	`theme_pain_pub()`	Publication-style ggplot theme.

Why the symmetric-midpoint anchor matters

For the pain–motor coupling analyses (Sprint 5; `R/26b_*` and `R/26c_*`), Never-DBS patients have no real "anchor event." Without a symmetric window, you cannot compute matched Δ-Δ contrasts across arms. The fix is to define each Never-DBS patient's anchor as the midpoint of their own follow-up — but the original code used:

```
first + as.numeric(difftime(last, first, units = "days")) / 2
```

When `first` is POSIXct, the addition adds *seconds*, not days. The midpoint collapses back to `first + ~6 minutes`. The fix is to cast first:

```
first <- as.Date(first)
last  <- as.Date(last)
first + as.numeric(difftime(last, first, units = "days")) / 2
```

This is now in `compute_symmetric_midpoint_anchors()` and unit-tested in `tests/testthat/test-anchor-midpo`

Main analysis — `R/build_*.R`

These scripts produce the manuscript's main and supplementary figures and tables. Each one is self-contained but `source()`s `R/helpers/pain_helpers.R`.

`build_fig5_and_tables.R`

Purpose: linear mixed-effects models (LMMs) for the Pre-DBS vs Post-DBS slope contrast (model A) and the Post-DBS vs Never-DBS slope contrast (model B). Produces Figure 5 (LMM predicted trajectories) and the slope-contrast tables.

Key calls:

```
m_A <- lme4::lmer(NP1PAIN ~ time_m * traj + (1 + time_m | PATNO),
                  data = df_A, weights = weight_sw_trim90)
ct_A <- emmeans::emmeans(m_A, ~ time_m * traj) |>
  pairs(reverse = TRUE)
```

Output: `outputs/figures/Figure5_lmm_pre_post_and_post_vs_never.png`, `outputs/tables/lmm_*_slope_c` and a cached RDS at `outputs/objects/fig5_lmm_fits.rds`.

Important note: `weights = weight_sw_trim90` are interpreted by `lme4` as precision weights (variance multipliers), not survey weights. For proper IPW interpretation, refit without weights and apply cluster-robust SE via `clubSandwich::vcovCR()` (Sprint 6 does this).

`build_gee_table3.R`

Purpose: generalised estimating equations with IPW weights for the NP1PAIN trajectory by phase. Produces Table 3.

Key calls:

```
m_base <- geepack::geeglm(
  NP1PAIN ~ time_m * traj,
  id = PATNO, data = df, corstr = "exchangeable",
  weights = weight_sw_trim90, family = stats::gaussian()
)
```

Output: `outputs/tables/gee_table3_base_vs_adjusted.csv`, `outputs/objects/gee_table3_fits.rds`.

Note: Sprint 6 re-fits with `corstr = "ar1"` as a sensitivity. The Pre-DBS × time interaction is sensitive to this choice (P = 0.079 □ P = 0.59); the Post-DBS × time interaction is robust.

`build_replication_figs.R`

Purpose: STROBE flow diagram (Figure 2) + wide-window LMM supplementary figure (FigureS5). Reads cohort counts directly from the data — no hardcoded n's after the cleanup.

`build_delta_24m_landmark.R`

Purpose: 24-month landmark Δ Pain stratified by baseline pain. Uses pre-window `[-6, 0]` and post-window `[18, 30]`. Welch's t-test per stratum, interaction P from a linear model.

`build_delta_matched_6_12mo.R`

Purpose: same idea at 6-month and 12-month landmarks, matched cohort. Produces Figure 6.

`build_stratified_delta_by_window.R`

Purpose: a sweep of Δ Pain across the −18 to +48 month window grid, stratified by baseline-pain tertile. Powers the supplementary visualisation of where in the trajectory the channeling pattern lives.

`build_alluvial_pain.R` / `build_alluvial_pain_matched.R`

Purpose: alluvial trajectories of NP1PAIN over visit bins. Uses LOCF/NOCB to fill within-patient gaps (documented in the figure caption). Produces 4 variants per cohort: absolute × 6 mo / 12 mo, proportional × 6 mo / 12 mo. Pick one main-text version; rest go to supplement.

Exploratory analyses — `R/25_*` and `R/26[abc]_*`

`R/25_genetics_arm_pain.R`

Four genetic / biomarker × DBS interactions on Δ Pain:

1. **PD-PRS** (constructed as allele-count sum over 55 NeuroChip variants; see note on rebuilding as a β-weighted score). All from the MJF Foundation supplementary `ppmi_database.db`.
2. **APOE-ε4** carrier vs non-carrier.
3. **CSF α-synuclein SAA** positivity (codes 1/2/3 all treated as positive).
4. **GBA** carrier status.

For each: `arm × stratum` interaction tested by likelihood-ratio + a forest plot. All four were null (LRT P 0.30 / 0.96 / 0.71 / 0.40) but the analyses are under-powered (n_DBS ≈ 50 per stratum); minimum detectable interaction effect ≈ 0.5 pain points on the 0–4 scale.

Caveat on PRS: the comment at line 119–122 explicitly notes the "allelic burden score" framing. Rename in any future submission if you do not implement a β-weighted version from Nalls 2019.

`R/26_pain_motor_coupling.R`, `R/26b_*.R`, `R/26c_*.R`

These three scripts test the *Pacheco-Barrios 2025* cross-sectional pain–motor association in PPMI (matched + full cohorts) and the longitudinal Δ-Δ Spearman coupling. `26b` uses the symmetric-midpoint anchor for Never-DBS controls (so visit grids are symmetric across arms); `26c` reframes pain as the outcome (rather than predictor).

Sprint 5 wraps the ordinal logistic regression with a manual Brant test and provides bootstrap Δρ confidence intervals; Sprint 5 also adds profile-likelihood and Firth-penalized CIs for small strata.

Sprint scripts — `sprints/sprint01-09.R`

The nine post-hoc robustness analyses, added 2026-05-19. Each is self-contained, seeded with `set.seed(20260519)`, and outputs a CSV table + (where appropriate) a PNG/PDF figure. Run sequentially via `make sprints`.

Script	What it tests	Headline number
`sprint01_negative_controls.R`	NP1HALL/URN/COG via same TOST framework	All 4 outcomes NI at ±1
`sprint02_anchor_sensitivity.R`	3 Never-DBS anchor schemes	All 3 NI at ±1
`sprint03_evalue_mnar.R`	E-value table + MNAR tipping-point	Tipping at k = ±1
`sprint04_nct_bootnet.R`	Network Comparison Test + bootnet	Late-post P = 0.050
`sprint05_bootstrap_brant_firth.R`	Bootstrap Δρ + Brant + Firth	PO holds; Δρ = –0.16
`sprint06_robust_ses.R`	Cluster-robust SE + GEE AR(1)	Pre-DBS slope sensitive to corstr
`sprint07_psm_diagnostics.R`	PS overlap + caliper sweep	c-stat = 0.885
`sprint08_competing_risk.R`	Fine-Gray competing risk	HR = 1.86 (1.28–2.69)
`sprint09_ledd_mediation.R`	ΔLEDD as mediator	matched ACME P = 0.69

Common structure (read once, then skim the rest):

1. `source("helpers/pain_helpers.R")` — load utilities
2. `set.seed(20260519)` — reproducibility
3. Define analysis window constants (`PRE_WIN`, `POST_WIN`, etc.)
4. Load cohort data via `load_*` helper
5. Compute per-patient summaries (Δ outcomes)
6. Run the focal statistical test
7. Save CSV + figure via `save_table()` / `save_fig_pub()`

Reading order: start with `sprint02_anchor_sensitivity.R` — it's the simplest and demonstrates the pattern. Then `sprint01`, `sprint07`, `sprint08`, `sprint09`. `sprint04` (NCT + bootnet) is the most complex and the only one with substantial runtime.

Python scripts — `scripts/*.py`

`scripts/_build_methods_figure.py`

Produces `Figure_methods_schematic.png/pdf` using matplotlib `FancyBboxPatch` and `FancyArrowPatch`. Layout is hand-tuned with hardcoded coordinates; non-fragile but verbose to modify. Tier rectangles (Primary / Secondary / Sensitivity / Exploratory) live in the `tiers` list at line 159.

`scripts/_build_docx_v9*.py`

Three closely-related scripts that build the manuscript Word document in place. **Critical**: these mutate the docx file. Always make a `.bak` before running on a real version. The `replace_paragraph_text()` helper preserves the paragraph node but drops all but the first run, which destroys inline formatting (bold/italic) — fine for captions, risky for body text.

scripts/build_callgraph.py

Parses every R / Python / ipynb file for `source(...)`, `readRDS(...)`, `read_csv(...)`, and `save_*(...)` calls. Builds two graphs:

- `callgraph.mmd` — full Mermaid graph (295 nodes; renders on GitHub for small repos, may exceed Mermaid's node limit for larger ones).
- `callgraph_overview.mmd` — 5-tier high-level summary that GitHub always renders interactively.
- `outputs/figures/Figure_callgraph.png/pdf` — static rendering via `networkx` + `matplotlib`.

Run after any analysis-script change: `python3 scripts/build_callgraph.py`.

scripts/build_sankey.py

Two-panel Sankey: Panel A = cohort flow (PPMI Curated □ idiopathic □ matched □ analytic), Panel B = analysis flow (tier □ script □ conclusion). Static PNG/PDF for the manuscript; interactive `outputs/figures/Figure_sankey.html` for the website.

scripts/build_dashboard.py

Generates the interactive HTML results dashboard at `docs/dashboard.html`. Reads every `sprint*.csv` under `outputs/tables/` and renders one Plotly panel per sprint, with KPI cards at the top. Uses Tailwind CSS (CDN-loaded) and meets WCAG 2.2 AA accessibility.

Tests — `tests/testthat/`

Three test files:

- `test-anchor-midpoint.R` — regression test for the POSIXct □ Date bug. The `compute_symmetric_midpoi` function must return midpoints in DAYS, not seconds.
- `test-helpers.R` — sanity checks on `OKABE_ITO`, `ARM_COLORS_OK`, `DAYS_PER_MONTH`.
- `test-time-pos-orientation.R` (to be added) — regression for the unsigned `time_pos` axis-inversion artifact.

Run all: `make tests`. CI runs them on every push.

Reproducibility infrastructure

Makefile

```
make env          restore renv + pip
make synth-data   regenerate synthetic PPMI fixture
make analysis     run primary/secondary/exploratory
make sprints      run sprint01-09
make figures      rebuild DAG, callgraph, Sankey, dashboard
make dashboard    rebuild only the interactive dashboard
make book         render the Quarto book
make tests        run testthat
make all          everything above
make clean        wipe rebuildable outputs
```

`Dockerfile`

`rocker/r-ver:4.5.1` base + system deps for `qgraph`, `glasso`, `lme4`, `geepack`, plus Python 3, Quarto. Use:

```
docker build -t ppmi-dbs-pain .
docker run --rm -v "$(pwd)":/work ppmi-dbs-pain make all
```

`.devcontainer/devcontainer.json`

Tells VS Code / GitHub Codespaces to build the Dockerfile and run `renv::restore()` + `pip install -r requirements.txt` on first start. A reviewer can click "Open in Codespaces" and have a full development environment in 5 minutes.

`.github/workflows/test.yml`

Runs on every push / PR:

1. Set up R 4.5.1.
2. Install all dependencies via `r-lib/actions/setup-r-dependencies`.
3. Run `testthat::test_dir("tests/testthat")`.
4. Lint with `lintr` (warnings only, non-blocking).
5. Set up Python 3.13 + install requirements.
6. Smoke-test `scripts/build_callgraph.py` and `scripts/build_sankey.py`.

Website setup

The Quarto book under `docs/` is the public-facing companion. Skeleton chapters live at `docs/*.qmd`; the book renders to `docs/_site/` and deploys to GitHub Pages via `.github/workflows/render.yml`.

To preview locally:

```
cd docs
quarto preview          # opens http://localhost:4444
```

To publish:

```
cd docs
quarto publish gh-pages
```

This deploys to `https://nielspac177.github.io/ppmi-dbs-pain/`.

The interactive dashboard (`docs/dashboard.html`) is included verbatim — no further build step.

Data flow at a glance

1. **Raw PPMI data** (user-supplied, in `config.yml`-defined paths) □
2. `load_*` **helpers** in `R/helpers/pain_helpers.R` □
3. **Per-script Δ / model / network computation** □
4. **CSV outputs** in `outputs/tables/` □
5. **Figures** in `outputs/figures/` (PNG + PDF) □
6. **Aggregated summaries** in `outputs/aggregated/` (PPMI-DUA-safe) □
7. **Interactive dashboard** in `docs/dashboard.html` □

8. **Manuscript** assembled via `scripts/_build_docx_v9_*.py`.

If you change *any* script and the headline number changes, the dashboard will reflect it on the next `make dashboard`, and CI will catch it on the next push.

Final words

This repo is a research artefact, not production software. Code is written for clarity and reproducibility, not raw performance. Conventions:

- All R scripts: `set.seed(20260519)` at the top; `source()` helpers; `dplyr::` namespace prefixes; `save_*()` outputs.
- All Python scripts: type hints on public functions; PEP-8; CLI entry via `if __name__ == "__main__"`.
- Tests are mandatory for any new helper function.
- Output files are *deterministic* given seeded random states; CI verifies by re-running the synthetic fixture pipeline end-to-end.

For questions, see CONTRIBUTING.md and REPRODUCE.md, or open an issue using the Replication template.